

# SKilling it!



Don't build agents. Build skills.

What skills are, when you need one, and how to write one well.

HI THERE

# Eva Mishor, PhD

Neuroscience ↔ Data science ↔ Building with coding agents daily

---



**429K**

lines of code shipped

**11**

projects built

**26 + 21**

skills + agents created

**4.1B**

tokens consumed

**823**

hours coding

**1,338**

sessions

---

Code: cumulative across 11 repos. Usage telemetry: ~3.5 months on file (Jan–Apr 2026).

WHY SKILLS EXIST

# Why we need skills at all

Every new session, you explain your process again. From scratch.

To transform **general-purpose AI models** into specialized, reliable workers that can interact with real-world tools, access specific company context, and automate complex, multi-step workflows.

# The Stack

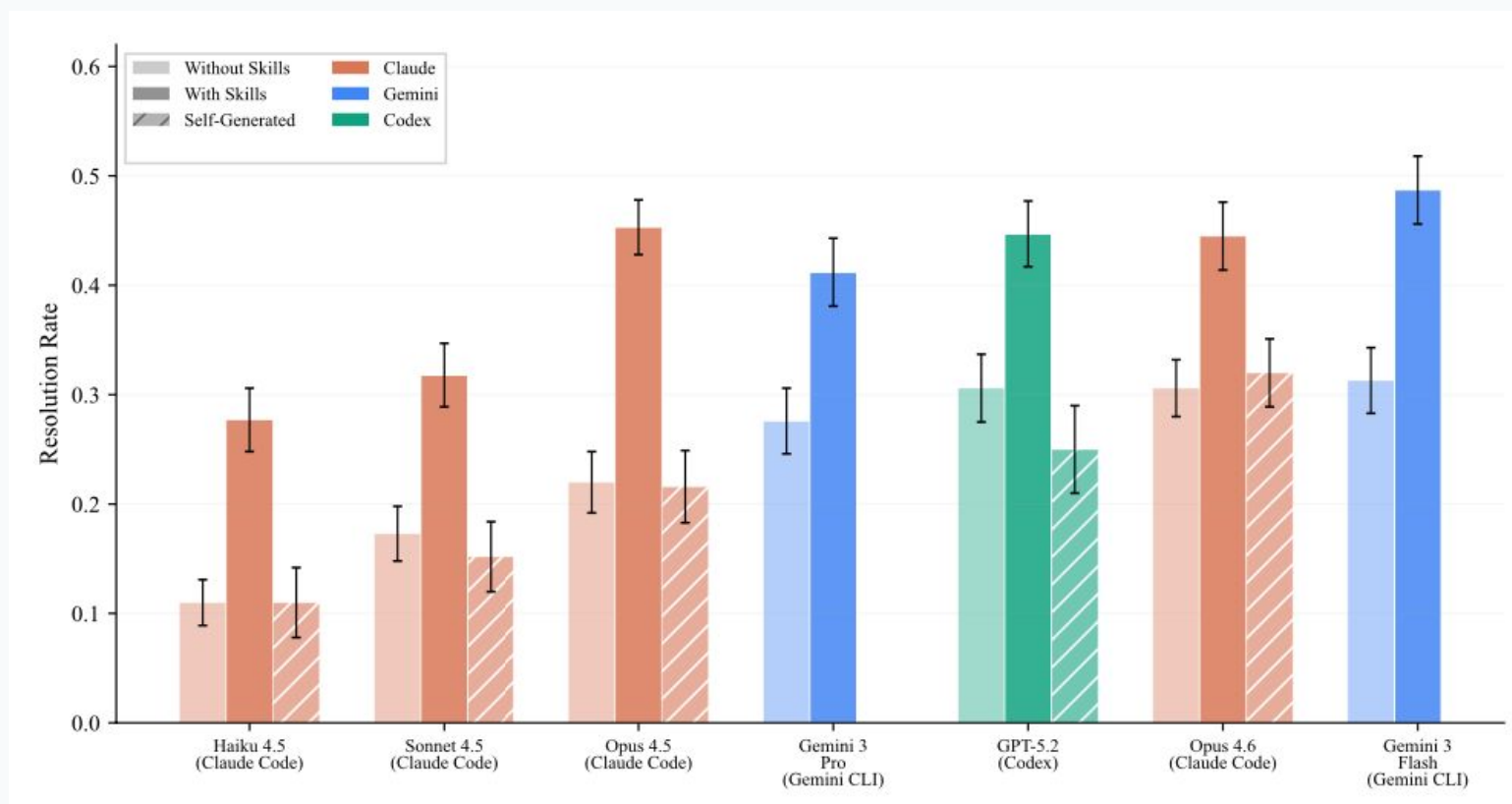
Don't tune the CPU. Don't rewrite the OS. Install the right apps.

|                |                        |                                                                        |
|----------------|------------------------|------------------------------------------------------------------------|
| <b>Skills</b>  | → the applications     | your/someone else's domain expertise                                   |
| <b>Harness</b> | → the operating system | Claude Code · Codex CLI · Gemini CLI · opencode · Github Copilot · etc |
| <b>Models</b>  | → the CPU              | Claude · GPT · Gemini · Kimi · MiniMax · etc                           |

# SkillsBench

Benchmarking How Well Agent Skills Work Across Diverse Tasks

• 7 models • 3 harnesses • 84 tasks • 11 domains • 7,308 trajectories



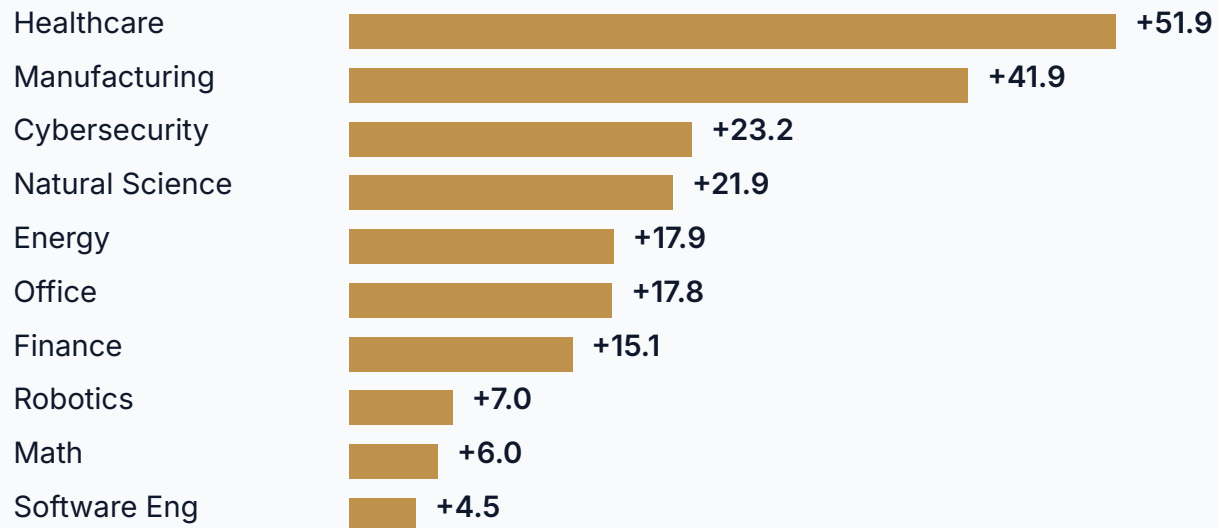
# Evaluating skills: what works

SkillsBench · 84 tasks · 7,308 trajectories · 7 model-harness configs · 2026

**+16.2pp**

average pass-rate gain with curated skills

## SKILLS LIFT, BY DOMAIN ( $\Delta$ pp)



# Why skills?

1

## Impose Determinism

Defined steps, gates, quality bars. The model can be creative within constraints.

+16.2pp avg lift, consistent across all 7 evaluated configs

2

## Give the Right Guidelines

Coding agnets know how to code. It doesn't know YOUR process, gotchas, conventions.

3

## Centralize Learning

Every mistake → a gotcha. Knowledge compounds across the team and over time.

4

## Scale Across the Team

Day-1 hire runs the same skill as a 5-year veteran. Commit to the repo.

5

## Context Efficiency

Progressive disclosure: load only what's needed.

2-3 focused skills > comprehensive docs by +18.6pp

# What is a skill?

A skill is a folder, not a prompt.

```
.claude/skills/my-skill/
├── SKILL.md      # core instructions
├── references/   # deep docs (on demand)
│   └── framework.md
├── assets/      # templates
│   └── report.md
└── scripts/     # executable tools
    └── run.sh
```

## PROGRESSIVE DISCLOSURE

|                    |                               |
|--------------------|-------------------------------|
| <b>SKILL.md</b>    | loads on trigger (>200 lines) |
| <b>references/</b> | loaded at specific steps      |
| <b>assets/</b>     | used when producing output    |
| <b>scripts/</b>    | run when agent needs tools    |



# When do you need a skill?

## You need a skill when:

- Same multi-step process, across sessions
- Decision points, gates, conditional logic
- Reference material too large for CLAUDE.md
- Others should run the same process
- Specific deliverables in specific formats

## You don't when:

- A CLAUDE.md instruction covers it
- It's a one-off task
- It's general knowledge the LLM has
- You're tempted to have Claude write it itself

# Three skills you can ship this week

we'll demo this

## BUSINESS PROCESS

### Weekly Metrics Digest

Pulls Stripe + Posthog data, drafts your Sunday metrics email.

name: weekly-metrics-digest  
description: Generate Monday metrics email from Stripe + Posthog. Triggers on 'metrics report', 'weekly numbers'.

## RUNBOOK

### Bug Investigation

Slack thread or alert in. Multi-step investigation. Structured report out.

name: bug-investigation  
description: Investigate a reported bug from Slack thread or alert. Triggers on 'investigate this', 'what's going on with X'.

## LIBRARY / API REFERENCE

### Tech Research

Researches current SOTA before you commit - linters, frameworks, libraries.

name: tech-research  
description: Research current state-of-the-art before implementing. Triggers on 'what's the best way to X', 'which library'.

# The SKILL.md file

```
---
name: deploy-checklist
description: "Run pre-deploy validation
for production releases. Triggers on
'deploy to prod', 'release checklist'."
allowed-tools: Bash, Read, Grep
---

# (markdown body — process flow,
#  decision logic, gotchas, gates)
```

**name** your slash command

/deploy-checklist. Kebab-case, lowercase.

---

**description** for agents, not humans

Trigger spec, not summary. Front-load use case + trigger phrases.

---

**allowed-tools** permissions

Restrict what Claude can do without asking. Read-only?  
No write access.

THE DEMO

# /tech-research

A general skill for getting up-to-date info from a model with stale training data.

The frontmatter:

```
---
name: tech-research
description: Research current state-of-the-art before implementing technical decisions. Use BEFORE recommending or implementing tooling (linters, formatters, test frameworks), libraries, dependencies, or architectural patterns. Triggers on questions like "what's the best way to...", "which library should I use for...", or any implementation planning in fast-moving domains (Python tooling, JS frameworks, cloud infrastructure, AI/ML). Prevents stale training-data recommendations.
---
```

# /tech-research

A general skill for getting up-to-date info from a model with stale training data.

SKILL.md

```
# Technical Research Skill

Research current SOTA before implementation to avoid stale recommendations.

> ## When This Triggers...

> ## Research

> ## Quick

> ## Examples

> ## Resources

## When This Triggers

1. **Tooling decisions**: linters, formatters, build tools, package managers
2. **Library selection**: frameworks, ORMs, testing libraries
3. **Architecture patterns**: that evolve rapidly (serverless, edge, AI/ML)
4. **"Best practice" questions**: where the answer changes yearly
```

# Progressive disclosure

A 2,000-line SKILL.md is a context bomb. Load only what you need.

## 1 System prompt

30–50 tokens

At startup, only the name + description load. Agent knows the skill exists, nothing more.

## 2 Trigger

On user prompt match

When the description matches, the full SKILL.md routing file is pulled into context.

## 3 Deep pipeline

Only when needed

references/, scripts/, assets/ load only at the specific step that needs them.

---

you don't pay the context bill unless you have to.

THE DEMO

# /tech-research

A general skill for getting up-to-date info from a model with stale training data.

SKILL.md

```
## Research Workflow

> ### Phase 0: Staleness Check (Self-Referential) ...

> ### Phase 1: Identify Domain Volatility ...

> ### Phase 1.5: Slow-Domain Exit Disclosure (Mandatory) ...

> ### Phase 2: Web Search (Mandatory for Fast-Moving Domains) ...

> ### Phase 2b: Community Recency Search (Mandatory for Highly Volatile Domains) ...

> ### Phase 3: Validate Against Multiple Sources ...

> ### Phase 4: Compare Training Data vs Reality ...
```

# /tech-research

A general skill for getting up-to-date info from a model with stale training data.

SKILL.md

```
## Research Workflow

> ### Phase 1: Initial Research
  ### Phase 1b: Initial Research (Mandatory for Highly Volatile Domains)

> ### Phase 2: Community Recency Search (Mandatory for Highly Volatile Domains)
  For domains marked as highly volatile in `references/fast-moving-domains.md` (currently: AI/ML Tooling, JS Build
  Tools, React Ecosystem), add **community searches**:

  ...

  Reddit:    "{topic} reddit {current_year}"
  Reddit:    "{topic} site:reddit.com"

> ### Phase 3: HN Search (Mandatory for Highly Volatile Domains)
  HN:        "{topic} site:news.ycombinator.com"
  Sentiment: "{topic} production experience {current_year}"

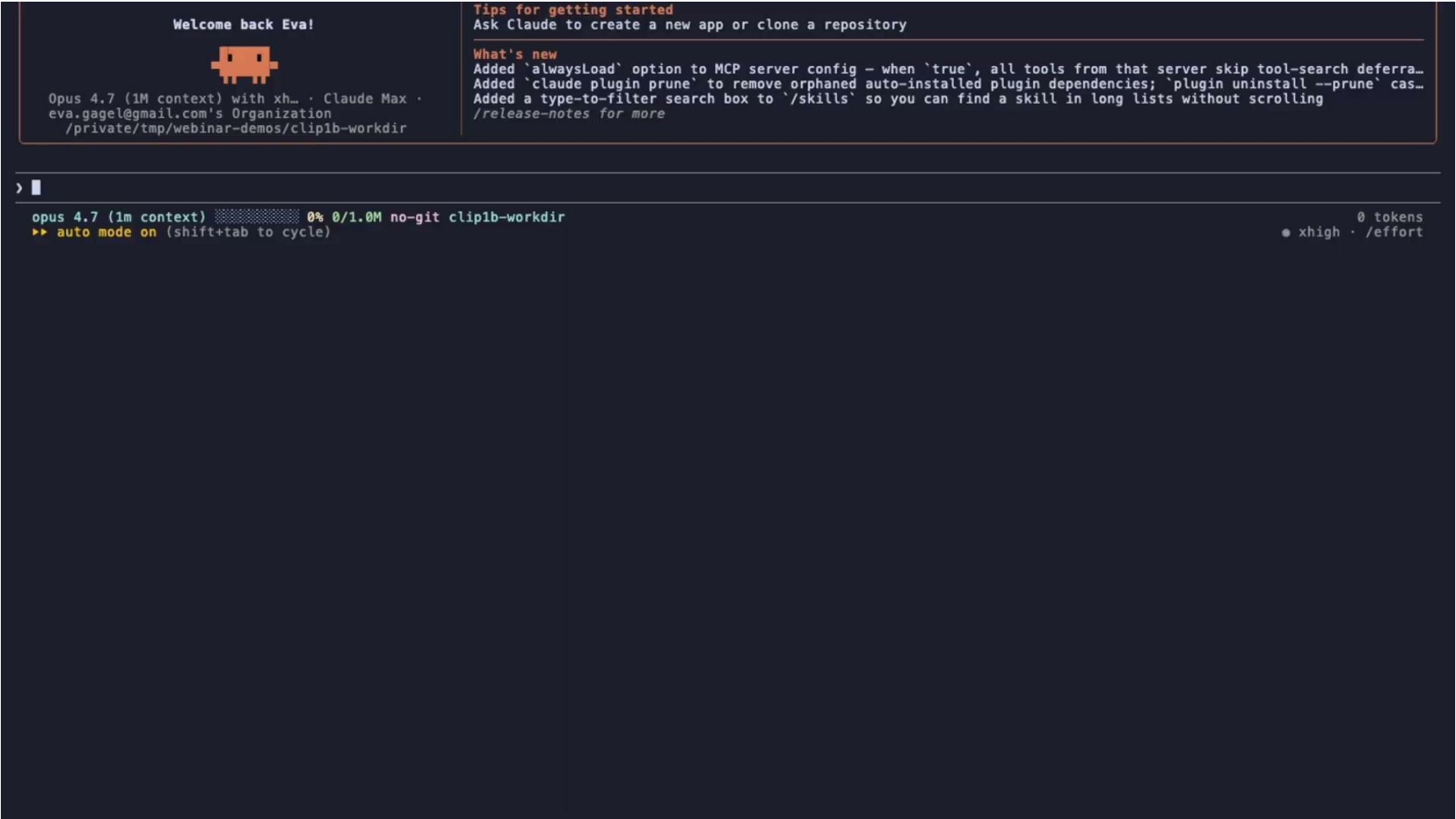
> ### Phase 4: Migration Search (Mandatory for Highly Volatile Domains)
  Migration: "switched from {tool} to reddit {current_year}"
  ...

> ### Phase 4: Compare Training Data vs Reality...
```



# /tech-research

A general skill for getting up-to-date info from a model with stale training data.



# /tech-research

A general skill for getting up-to-date info from a model with stale training data.

Opus 4.7 (1M context) with xhigh effort · Claude Max  
~/Documents/Code/my-claude-skills

> █

opus 4.7 (1m context) ██████████ 0% 0/1.0M main (9) my-claude-skills0 tokens  
➤ auto mode on (shift+tab to cycle)1 MCP server needs auth · /mcp

| Cond                        | Wall | Tokens | Tool calls | Quality | Tokens/quality point |
|-----------------------------|------|--------|------------|---------|----------------------|
| A. Opus, no skill (+year)   | 29s  | 13,215 | 0          | 1/3     | 13,215               |
| B. Opus, no skill (no year) | 32s  | 13,447 | 0          | 0/3     | ∞                    |
| C. Opus + /tech-research    | 81s  | 34,511 | 8          | 3/3     | 11,504               |
| D. Sonnet + /tech-research  | 86s  | 23,355 | 12         | 3/3     | 7,785                |

# Skills can replace model size

Same SkillsBench evaluation. Two configurations.

HAIKU 4.5 + SKILLS

**27.7%**

the smaller, cheaper model but equipped

OPUS 4.5 (NO SKILLS)

**22.0%**

the bigger, costlier model running cold

A skill on a smaller model beats a larger model running cold.

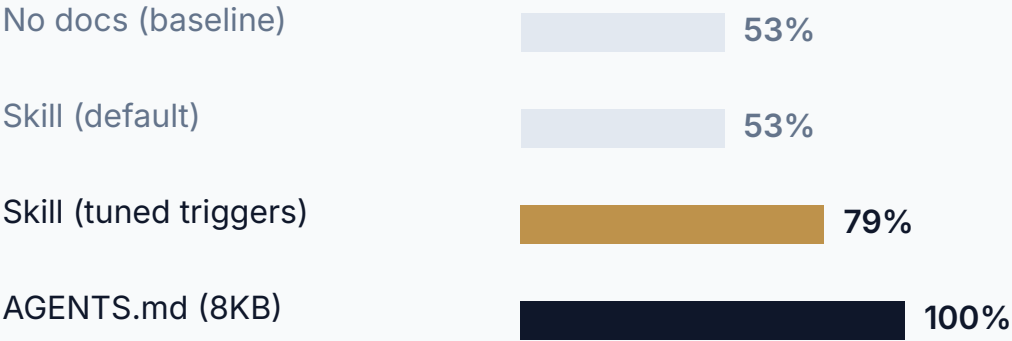
(SkillsBench Finding 7)

THE DOWNSIDE

# How skills can break

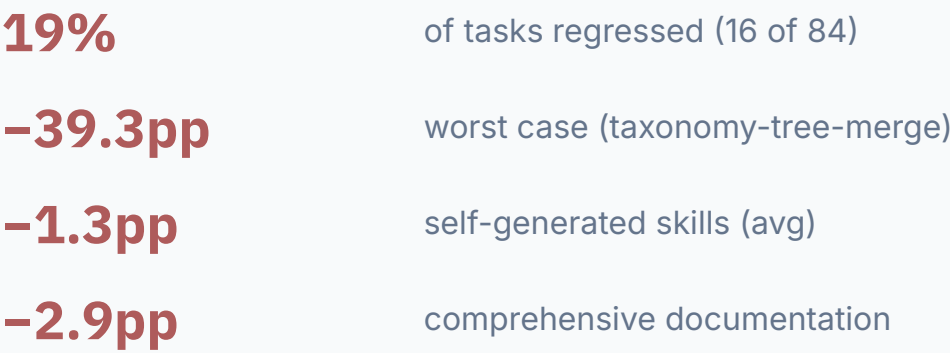
Sometimes a 'dumb' static file beats a sophisticated skill.

VERCEL · NEXT.JS 16 DOCS · SAME CONTENT, THREE INJECTIONS



Default skill = baseline. Tuned skill < AGENTS.md.  
Skills require a decision to invoke. Static facts don't need decisions — they need to be present.

SKILLSBENCH · 84 TASKS · REGRESSION CASES



- WHY IT HAPPENS
- Tasks the model already aces - guidance becomes noise.
  - Models can identify need for a procedure, but generate imprecise ones.
  - Codex agents acknowledge skills, then ignore them. (SkillsBench 4.1.2)

# Four rules that matter

1

## Build a gotchas section

Highest-signal content = what goes wrong. Every mistake → add it. After 10 sessions this is your most valuable section.

2

## Avoid railroading

Principles and constraints, not rigid scripts. Brittle instructions break the moment edge cases appear.

3

## One type, one skill

If it straddles two distinct workflows, split it. Confused skills do neither well.

4

## Evals

Treat skills like code. Live session → first draft → find gaps (Red) → fix (Green) → refactor → ship → evals → improve

# When to use what

You have four levers. Mix & Match each to its job.

## Hooks

Deterministic action

MECHANISM

Harness-executed, runs on event

CONTENT

Shell commands on tool/session lifecycle

IDEAL FOR

"Auto-format after every Edit" /  
"Block rm -rf in Bash"

## AGENTS/CLAUDE.md

Passive context

MECHANISM

Always-on, persistent

CONTENT

Facts, frameworks, indices (compressed <8KB)

IDEAL FOR

"We use Next.js 16 APIs"

## Skills

Procedural context

MECHANISM

On-demand, progressive disclosure

CONTENT

Multi-step workflows, gates, scripts

IDEAL FOR

"Run the production release checklist"

## MCP

Data context

MECHANISM

External API bridging

CONTENT

Live databases, 3rd-party state

IDEAL FOR

"Fetch ticket status from Jira"

Choose hard when possible - code is better than LLM-dependent invocation .

# Sharing & installing

| SCOPE      | PATH                     | WHO GETS IT            |
|------------|--------------------------|------------------------|
| Personal   | ~/.claude/skills/<name>/ | Just you, all projects |
| Project    | .claude/skills/<name>/   | Everyone on this repo  |
| Enterprise | Managed settings         | All org users          |

SHARE VIA GITHUB

```
claude install-skill https://github.com/you/repo/tree/main/skills/my-skill
```

THE ECOSYSTEM TODAY (SkillsBench, Mar 2026)

47,150 unique skills · 84,192 collected · peak 18,904 created in a single day (Jan 22, 2026)

# At-a-glance

1

## A skill is a folder (app)

SKILL.md + references + assets + scripts

2

## Progressive disclosure

SKILL.md is the router. References are the depth. keep it simple.

3

## Description triggers the skill

Write it for agents, not for humans. Measure invocation.

4

## Gotchas are your best content

Build them from real failures over time → learn from experience

5

## Do the work first

Live session → red/green/refactor → ship.

---

**START HERE** Open your favorite coding agent → do the work → /branch off → /skill-creator → evals → add a gotcha when it fails

**THEN** /skill-creator built-in evals to measure if it's actually helping. keep improving as you go → gotchas

---

**Golden rules:** Curate, don't generate. 2–3 focused skills > one giant skill. Measure performance. Discard what doesn't produce value. Beware of open-source skills (security risk). Don't skill what is already known. Heanesses and models behave differently, measure.



# SKilling it!

Thank you

Go home and Build a skill.  
Today.



LinkedIn:  
[@eva-mishor](#)

**ITP** IDEA  
TO PRODUCT



**Idea-to-Product.com**

# Resources

[Anthropic — How We Use Skills](#)

[Claude Code Documentation — Skills](#)

[The Complete Guide to Building Skills](#)

[Anthropic — Equipping agents for the real world \(Oct 2025\)](#)

[SkillsBench — empirical evaluation](#)

[Vercel — AGENTS.md outperforms Skills in our evals](#)

[Agent Skills Open Standard](#)

[eval-compare — the demo skill, fork it](#)

Built into Claude Code: [/skill-creator](#) · [/superpowers:writing-skills](#)